

Optimización por Enjambre de Partículas (PSO) en la selección de características para mejorar la Eficiencia computacional en la detección de intrusiones en entornos de Tecnologías de Operación, 2026

L. J. Villegas ¹, ,*

¹ Facultad de Ingeniería Eléctrica y Electrónica, Universidad Nacional de Ingeniería, Av. Túpac Amaru 210 Rímac, Lima, 15333, Perú

*Corresponding author: L.J. Villegas (luis.villegas.n@uni.pe)

Abstract

La convergencia de redes en entornos de Tecnologías de Operación (OT) ha incrementado críticamente la exposición a Ataques de Inyección de Datos Falsos (FDIA). Sin embargo, la "maldición de la dimensionalidad" inherente al tráfico de protocolos industriales genera sobrecargas computacionales que dificultan el despliegue de Sistemas de Detección de Intrusiones (IDS) en nodos de borde (*Edge*) con recursos físicos restringidos. Este estudio tiene como objetivo mejorar la eficiencia computacional en la detección de intrusiones en ecosistemas OT mediante la minimización del espacio dimensional. Para ello, se propone una arquitectura de envoltura (*wrapper*) que integra la Optimización por Enjambre de Partículas Binario (BPSO) como algoritmo de selección de características y un modelo *Random Forest* (RF) como clasificador. Evaluado sobre un conjunto representativo de tráfico Modbus/TCP ($N = 41,535$), los resultados empíricos demuestran que el enfoque BPSO-RF alcanzó un ratio de compresión del 55.47%, reteniendo 57 de las 128 características originales. Esta reducción dimensional logró una disminución del 48.64% en el consumo pico de memoria RAM (2.37 MB) y un 11.01% en la latencia de inferencia (0.0239 ms), experimentando una degradación predictiva marginal de apenas 0.51% en el F1-Score (0.9166). Se concluye que la arquitectura propuesta viabiliza empíricamente el despliegue de modelos *Lightweight* en dispositivos perimetrales, consolidando una topología *Cloud-Edge* donde la alta carga computacional de la optimización metaheurística se gestiona de forma desconectada (*Offline*) para garantizar una respuesta operativa en tiempo real.

Palabras clave: Ciberseguridad industrial, Tecnologías de Operación (OT), Optimización por enjambre de partículas (PSO), Selección de características, Edge computing, Detección de intrusiones.

Ideas destacadas

- BPSO redujo la dimensionalidad del tráfico Modbus/TCP en un 55.47%.
- El F1-Score mantuvo una alta robustez predictiva (0.9166) ante ataques FDIA.
- Consumo pico de RAM minimizado a 2.37 MB con latencia ultrabaja de 0.0239 ms.
- El modelo delega la optimización a la nube y ejecuta inferencia ligera en el Edge.

1 Introducción

La integración del paradigma de la Internet de las Cosas (IoT) en los Sistemas de Control Industrial (IACS) ha transformado las arquitecturas convencionalmente aisladas en ecosistemas hiperconectados que componen los entornos de Tecnologías de Operación (OT) y la Internet Industrial de las Cosas (IIoT). Si bien esta interconexión elimina barreras informáticas y optimiza procesos críticos, también ha expandido drásticamente la superficie de ciberataques contra estas infraestructuras vitales, exponiéndolas a vulnerabilidades externas. Como señalan Francis et al. (2026), las aplicaciones operacionales modernas —tales como la manufactura inteligente y las redes eléctricas inteligentes (Smart Grids)— generan volúmenes masivos de datos a alta velocidad. En consecuencia, garantizar una comunicación segura bajo requerimientos ineludibles de procesamiento en tiempo real se ha convertido en un desafío crítico de la ciberseguridad industrial contemporánea.

A pesar de la urgencia por asegurar estos entornos, los Sistemas de Detección de Intrusiones (IDS) dinámicos basados en aprendizaje automático enfrentan severas limitaciones de aplicabilidad práctica. Como detallan Gupta et al. (2025), el

desafío principal radica en la "alta dimensionalidad de los datos" inherente al tráfico de red industrial, lo cual genera modelos computacionalmente pesados que exigen una alta capacidad de memoria y procesamiento. Este costo computacional excede las restricciones inherentes de recursos (CPU y RAM) impuestas por el hardware perimetral (Edge), en donde operan los dispositivos industriales. Procesar estos espacios de características dimensionalmente altos provoca una elevada latencia de inferencia, lo que impide detectar cargas útiles maliciosas con los tiempos de respuesta críticos que exigen las operaciones industriales para evitar disrupciones críticas en la infraestructura (Tiwari et al., 2024). Para superar estas limitaciones, la literatura reciente establece que la reducción de dimensionalidad mediante la selección de características es el paso metodológico fundamental previo a la clasificación, ya que disminuye directamente la sobrecarga computacional sin sacrificar la asertividad. Dentro de esta línea, los algoritmos de Inteligencia de Enjambre (Swarm Intelligence), y específicamente la Optimización por Enjambre de Partículas (PSO), han emergido como el enfoque de vanguardia. Como argumenta Shan (2025), estas técnicas metaheurísticas logran un equilibrio eficiente entre la exploración global y la explotación local para automatizar la

selección de los subconjuntos de características más discriminativos. No obstante, a pesar de las ventajas teóricas documentadas, persiste la necesidad de evaluar rigurosamente el impacto tridimensional de estos algoritmos —en términos de espacio, tiempo y hardware— mediante experimentación controlada con datos reales de tráfico OT (Zayed et al., 2026; Mohammed et al., 2026).

En este contexto, la presente investigación adopta un enfoque empírico con el objetivo de implementar la Optimización por Enjambre de Partículas (PSO) para la selección de características a fin de mejorar la eficiencia computacional —reducción de dimensionalidad, complejidad temporal y huella de recursos— en la detección de intrusiones en entornos de Tecnologías de Operación. Para lograr este objetivo, las principales contribuciones de este artículo se resumen de la siguiente manera:

- Implementación de un motor de selección basado en PSO enfocado exclusivamente en la retención de características críticas para tráfico OT real.
- Evaluación empírica tridimensional que mide simultáneamente la reducción de dimensionalidad, el consumo pico de memoria volátil (RAM) y la latencia de inferencia, para demostrar la viabilidad del modelo propuesto bajo requerimientos de respuesta en tiempo real propios de la industria.

2 Revisión de literatura

El aseguramiento de los entornos de Tecnologías de Operación (OT) y de la Internet Industrial de las Cosas (IIoT) ha impulsado esfuerzos recientes para mitigar las vulnerabilidades en infraestructuras con recursos estrictamente limitados. La literatura actual concuerda en que la alta dimensionalidad de los datos de red genera una sobrecarga computacional que paraliza a los Sistemas de Detección de Intrusiones (IDS) tradicionales, impidiendo su despliegue en la frontera de la red (Edge). Para resolver este problema, Tiwari et al. (2024) demostraron la necesidad de crear arquitecturas con un payload mínimo, empleando técnicas de optimización y cuantificación para reducir la latencia de respuesta en un 25% en dispositivos perimetrales. Bajo esta misma línea, Gupta et al. (2025) abordaron la detección dinámica de malware en sistemas IIoT con restricciones de energía mediante un marco híbrido (GWPSO-GAMD), logrando reducir empíricamente el uso de la CPU en un 47%, la huella de memoria en un 57% y el tiempo de entrenamiento en un 67%. Sin embargo, como señalan Gaber et al. (2023), aunque la extracción de características críticas mediante algoritmos bioinspirados disminuye los costos de procesamiento, gran parte de las implementaciones tempranas en IIoT se limitaron a evaluar métricas tradicionales de rendimiento predictivo, dejando un vacío en la medición exhaustiva de la eficiencia real del hardware.

Para superar la crisis de la dimensionalidad, las estrategias metaheurísticas basadas en la Inteligencia de Enjambre (Swarm Intelligence) se han posicionado como la solución algorítmica ideal, destacando específicamente la Optimización por Enjambre de Partículas (PSO). El poder del PSO radica en su capacidad matemática para equilibrar dinámicamente la

exploración global y la explotación local en espacios de búsqueda complejos. Por ejemplo, Benmalek y Seddiki (2025) evidenciaron que los modelos mejorados con PSO, al integrarse con clasificadores como CatBoost, logran una reducción drástica de la sobrecarga computacional mientras superan a los métodos del estado del arte en la clasificación de ataques IoT modernos. No obstante, las metodologías difieren en su aplicación central; Alsalamah e Ismail (2025) propusieron un marco multiobjetivo (MOOIDS-IoT) que delega la selección de características a un Algoritmo Genético (GA) y reserva una variante de PSO únicamente para el ajuste de hiperparámetros, buscando un equilibrio entre la velocidad de convergencia y la complejidad del modelo. En contraste, Zayed et al. (2026) optaron por el Algoritmo de la Luciérnaga (FA) para optimizar un marco híbrido profundo (CNN-RNN), lo que demuestra que, independientemente de la técnica de enjambre elegida, la selección automatizada de características es un prerrequisito ineludible para viabilizar el aprendizaje automático en escenarios de tiempo real.

La aplicación de estas estrategias metaheurísticas en entornos verdaderamente críticos representa la frontera empírica actual de la ciberseguridad industrial. Abordando ecosistemas de redes eléctricas inteligentes (Smart Grids), Mohammed et al. (2026) diseñaron el modelo IG-APSO-DNN para combatir los Ataques de Inyección de Datos Falsos (FDIAs), utilizando un PSO Adaptativo (APSO) que logró reducir la dimensionalidad original del conjunto de datos en más de un 75%, manteniendo una precisión de detección superior al 97.90%. Por su parte, evaluando el despliegue directo en dispositivos con capacidades restringidas, Nasir et al. (2026) introdujeron HED-ID, un marco que integra una arquitectura temporal bidireccional (S-BiGRU) optimizada mediante la Optimización del Lobo Gris (GWO). El análisis empírico de Nasir et al. (2026) en entornos simulados tipo Edge es uno de los más rigurosos de la literatura reciente, reportando una viabilidad operativa sobresaliente con una latencia de inferencia ultrabaja de 18 a 22 milisegundos y un consumo de memoria restringido a un rango de 92 a 115 MB.

A pesar de estos avances significativos, una brecha de investigación recurrente es la transición metodológica desde los entornos IoT generales hacia los ecosistemas críticos. Muchas de las soluciones planteadas para IoT/IIoT no son directamente extrapolables a entornos puramente de Tecnologías de Operación (OT), como sistemas SCADA o PLCs, debido a sus intolerancias estructurales a fallos y retrasos. Además, aunque estudios recientes han comenzado a reportar métricas de latencia (Nasir et al., 2026) o de reducción de sobrecarga computacional (Benmalek & Seddiki, 2025), persiste un vacío en la medición simultánea e integral de la eficiencia tridimensional (espacio, tiempo y recursos de hardware) bajo un enfoque exclusivo de PSO en redes OT. Adicionalmente, múltiples enfoques híbridos (como los de Alsalamah & Ismail, 2025) relegan al algoritmo PSO a tareas de optimización de hiperparámetros en lugar de utilizarlo como motor principal para la selección de características, lo que incrementa la complejidad de la arquitectura sin garantizar tiempos de inferencia viables para la industria. Esta limitación consolida la necesidad fundamental de nuestra investigación, la cual justifica el uso directo del algoritmo PSO para la

selección de características como mecanismo central para construir un modelo verdaderamente ligero (Lightweight). Nuestro enfoque metodológico es pertinente y necesario porque no solo busca detectar intrusiones con alta precisión, sino que ancla su valor científico en la medición rigurosa de la

eficiencia computacional —evaluando el ratio de compresión, la latencia temporal y el consumo pico de memoria—, garantizando así su aplicabilidad técnica y sostenible en las Tecnologías de Operación.

Tabla 1
Resumen de trabajos relacionados

Referencia	Objetivo	Selección de Características y Modelo	Dataset / Entorno	Resultados Principales	Limitaciones Clave del Modelo
Francis et al. (2026)	Revisión sistemática de 36 estudios sobre IDS basados en metaheurísticas para entornos IIoT.	Múltiples (Inteligencia de Enjambre en un 69.4%) / Múltiples (RF, CNN, DNN).	IIoT, IoT / Múltiples (SWaT, WUSTL-IIOT, NSL-KDD).	Combinaciones con PSO superan el 99.7% de precisión. Hallazgo crítico: Solo el 19.4% evalúa la complejidad temporal y el 66.7% ignora por completo la eficiencia computacional.	Falta casi absoluta de validación real en hardware físico limitado (<i>Edge devices</i>); los modelos actúan como "cajas negras" sin explicabilidad; y se ignoran ataques específicos de protocolos industriales (ej. Modbus/TCP)
Gupta et al. (2025)	Detectar malware en redes IIoT con recursos limitados usando el framework GWPSO-GAMD.	Híbrido GWPSO (GWO + PSO) / GNN (GCN + GAT).	Móviles, Edge, IIoT / CIC-MalDroid-2020, CIC-MalMem-2022.	10 características retenidas. CPU -47%, RAM -57%, tiempo de entrenamiento -67.5%. Inferencia 3x más rápida. Precisión máxima: 98.41%.	Enfocado en malware móvil (Android), no aborda ataques de tráfico de red en protocolos puros de Tecnologías de Operación (OT/SCADA).
Nasir et al. (2026)	Proponer HED-ID, un IDS explicable y eficiente para despliegue directo en el <i>Edge</i> .	Optimizador del Lobo Gris (GWO) / S-BiGRU.	Cloud, Edge / CICIDS-2017, ToN-IoT, UNSW-NB15.	Latencia: 18-22 ms. RAM: 92-115 MB. Precisión: 93.8% (Edge). Integra explicabilidad (SHAP) con bajo impacto.	El consumo de RAM (hasta 115 MB) es prohibitivo para nodos IoT ultra-limitados. Baja tasa de detección en ataques minoritarios.
Mohammed et al. (2026)	Detectar inyección de datos falsos (FDIAs) en <i>Smart Grids</i> mitigando la complejidad dimensional.	Híbrido IG + APSO / DNN.	Smart Grids (OT) / ICS Cyber Attack.	Dimensionalidad reducida >75% (117 a 25). Tiempo de optimización: 400-836s. Precisión: 97.90%, Tasa de falsas alarmas (FAR): 2.11%.	La combinación IG+APSO introduce alta sobrecarga computacional, limitando seriamente su viabilidad para despliegues en tiempo real.
Tiwari et al. (2024)	Detectar intrusiones en redes IIoT con un <i>payload</i> mínimo para reducir latencia.	PSO / MARS y GAM.	IIoT (Edge) / WUSTL-IIOT-2021.	Latencia -25% (7.04s). 10 características retenidas. Precisión: 100%.	Los autores declaran no haber evaluado el consumo energético, uso de memoria (RAM) ni procesamiento (CPU).
Benmalek y Seddiki (2025)	Desarrollar un IDS para IoT basado en anomalías con baja sobrecarga computacional.	PSO+RF / CatBoost, SVM, LSTM, CNN.	IoT / RT_IoT2022.	Tiempo de entrenamiento reducido drásticamente (ej. LSTM -55.5%, SVM de 342s a 14.6s). Precisión máxima: 99.85% (CatBoost).	Baja detección en ataques minoritarios. No evalúa el consumo de recursos físicos (RAM/CPU en <i>Edge</i>).
Alsalamah e Ismail (2025)	Desarrollar el marco MOOIDS-IoT para ciberseguridad ligera y en tiempo real.	GA (Selección) + MOO-PSO (Hiperparámetros) / XGBoost, RF.	IoT / CICIoT2023, NSL-KDD.	Reducción a 12-20 características. Tiempo de entrenamiento: 28.2s (XGBoost). Exactitud: 98.38% (CICIoT2023)	Uso de GA en lugar de PSO para selección. Fase de optimización excesivamente lenta (hasta 4 horas), impidiendo el reentrenamiento en tiempo real.

3 Metodología

3.1 Fundamentación del Método y Diseño Experimental

La presente investigación adopta un enfoque cuantitativo, de nivel explicativo y un diseño cuasi-experimental. Se estructuró el experimento para evaluar rigurosamente los efectos de la reducción de dimensionalidad estableciendo una comparación directa entre dos configuraciones: un modelo base (Grupo de Control), el cual fue entrenado con la totalidad de las características del tráfico de red, frente a un modelo optimizado (Grupo Experimental), entrenado exclusivamente con el subconjunto de características extraídas por el algoritmo de Optimización por Enjambre de Partículas (PSO). Este rigor empírico es imperativo en la ciberseguridad industrial, dado que evaluar únicamente la eficacia predictiva de los Sistemas de Detección de Intrusiones (IDS) resulta insuficiente; es indispensable cuantificar experimentalmente si la manipulación de la variable independiente, constituida por la optimización metaheurística BPSO (ver Sección 4.1) generan mejoras directas en la huella de hardware del entorno perimetral (Tiwari et al., 2024; Nasir et al., 2026).

3.2 Descripción del Dataset

La validación experimental empleó el *Industrial Control System (ICS) Cyber Attack Dataset* (Power System Dataset), desarrollado por la Universidad Estatal de Misisipi y el Laboratorio Nacional de Oak Ridge. El entorno emula una infraestructura SCADA y de red eléctrica inteligente (Smart Grid) compuesta por dos generadores, múltiples líneas de transmisión y cuatro disyuntores controlados por relés (IEDs) bajo esquemas de protección de distancia (Borges Hink et al., 2014).

El conjunto captura 37 escenarios operativos, abarcando operaciones normales, perturbaciones naturales (ej. cortocircuitos) y ciberataques. Las amenazas principales incluyen inyecciones de datos falsos (FDIA) orientadas a la manipulación de corriente y voltaje, inyección de comandos de disparo remoto y alteraciones no autorizadas de configuración. Cada registro consta de un vector de 128 características. De estas, 116 son mediciones físicas extraídas de cuatro Unidades de Medición Fasorial (PMUs, 29 variables cada una), e incluyen magnitudes y ángulos de fase (voltaje/corriente), frecuencia, tasa de cambio de frecuencia (dF/dt) e impedancia aparente. Las 12 características restantes provienen de la capa cibernética y de monitoreo de la subestación, integrando indicadores lógicos derivados de registros (*logs*) de paneles de control y alertas del sistema de detección de intrusiones Snort (Borges Hink et al., 2014).

Para viabilizar el costo computacional de la metaheurística BPSO, se utilizó el subconjunto estandarizado del 1% provisto por los autores originales. Este muestreo estratificado preserva la representatividad estadística y la proporción original de las clases, mitigando sesgos de aprendizaje.

En consecuencia, el corpus experimental utilizado en esta investigación comprende un total de 41,535 instancias, distribuidas exactamente en 22,019 registros de tráfico normal y 19,516 registros de ataques FDIA. Finalmente, para

garantizar la validez de las métricas predictivas, este espacio se particionó aislando un conjunto de prueba (*test set*) independiente de 8,307 instancias no vistas previamente por el modelo.

3.3 Procedimiento Experimental

La ejecución de la experimentación se desarrolló en un entorno de alto rendimiento *Cloud-like* (Google Colab) utilizando Python 3 y librerías de aprendizaje automático estandarizadas (scikit-learn, imbalanced-learn, pyswarms). Para garantizar la reproducibilidad de los resultados, se estableció una semilla pseudoaleatoria global a lo largo de todo el código. El experimento se estructuró operativamente en cuatro fases secuenciales:

Fase 1: Preprocesamiento y Aislamiento Estricto

El experimento inició con la depuración del conjunto de datos mediante la eliminación de las columnas con varianza nula, descartando atributos estáticos que no aportan valor discriminativo al modelo (Mohammed et al., 2026). Tras realizar una imputación por la mediana para manejar valores faltantes sin descartar registros, el conjunto de datos se dividió rigurosamente de forma estratificada en un 80% para entrenamiento y un 20% para prueba.

Con el objetivo de prevenir la fuga de datos, las transformaciones estadísticas posteriores se aislaron en un procedimiento de dos pasos. Primero, los parámetros matemáticos de ajuste —específicamente, los límites del Rango Intercuartílico (IQR) para la winsorización de valores atípicos y los valores umbral para la normalización Min-Max— se calcularon exclusivamente a partir de la distribución del conjunto de entrenamiento. Inmediatamente después, estos parámetros precalculados se utilizaron para transformar y proyectar los datos del conjunto de prueba, asegurando un alineamiento dimensional perfecto sin permitir que el modelo asimile distribuciones estadísticas futuras.

Este proceso de normalización ajustó las características numéricas al rango continuo $[0, 1]$ y se formaliza mediante la Ecuación 1:

$$x' = \frac{x - x_{min}}{x_{max} - x_{min}} \quad (1)$$

Donde x es el valor original de la característica, y x_{min} y x_{max} representan los valores mínimo y máximo de dicha característica en el conjunto de datos de entrenamiento, respectivamente.

Finalmente, esta fase concluyó con la aplicación de la Técnica de Sobremuestreo de Minorías Sintéticas (SMOTE) de forma exclusiva sobre la partición de entrenamiento para balancear las clases, evitando así sesgos algorítmicos hacia el volumen masivo del tráfico industrial benigno.

A diferencia de aproximaciones recientes en la literatura que emplean imputación basada en K-Vecinos Más Cercanos (KNN) (Shees et al., 2024), en esta investigación se optó deliberadamente por la imputación mediante la Mediana. Esta decisión arquitectónica se fundamenta en los requerimientos de la fase de inferencia perimetral: mientras que algoritmos como KNN exigen mantener el corpus de entrenamiento en memoria para calcular distancias incrementando prohibitivamente la

latencia y la huella de RAM, la imputación por la mediana presenta una complejidad $\mathcal{O}(1)$, siendo más adecuado para preservar la naturaleza Lightweight del modelo en entornos OT limitados.

Fase 2: Línea Base (Grupo de Control)

Una vez aislado y preprocesado el dataset, se procedió a instanciar y entrenar el clasificador Random Forest (RF) utilizando la dimensionalidad original completa (128 características). Para garantizar su reproducibilidad exacta, el ensamble se configuró explícitamente con 100 árboles de decisión, una semilla global pseudoaleatoria y ejecución en paralelo, empleando todos los núcleos lógicos disponibles para maximizar la eficiencia del procesamiento. Durante la fase de inferencia sobre el conjunto de prueba, se capturaron las métricas iniciales de desempeño predictivo y eficiencia. Para asegurar el rigor metodológico, la latencia de inferencia y la huella de hardware se monitorizaron mediante perfilamiento en tiempo de ejecución, utilizando contadores de alta resolución y librerías de gestión de memoria asíncrona. Este Grupo de Control se establece como la referencia de rendimiento absoluto, permitiendo cuantificar empíricamente el compromiso (*trade-off*) entre la asertividad del modelo y la ganancia en eficiencia computacional tras aplicar la reducción dimensional.

Fase 3: Optimización Heurística (Variable Independiente)

En esta etapa, se ejecutó el algoritmo de Optimización por Enjambre de Partículas Binario (BPSO) —cuya fundamentación matemática y arquitectónica se detalla en la Sección 4— para la selección automatizada de las características más discriminativas. El optimizador estocástico actuó como un método de envoltura (*wrapper*), configurado con una población de 20 partículas y un límite de 30 iteraciones. Para mitigar el riesgo de sobreajuste (*overfitting*) durante la evaluación de la función de aptitud interna, el BPSO utilizó un sub-clasificador RF ligero e implementó una validación interna mediante una división estratificada tipo Hold-out (80/20). Este mecanismo garantiza que cada partícula sea evaluada sobre datos no vistos durante su ajuste interno, mitigando el sobreajuste en la selección de características. A través de este mecanismo, la variable independiente abstraigo el espacio de búsqueda para retener un vector óptimo y compacto de características.

Fase 4: Evaluación Experimental Final

Para concluir el diseño experimental, se procedió a efectuar el reentrenamiento del modelo de clasificación RF, configurado con hiperparámetros idénticos a los del Grupo de Control, pero ingiriendo en esta ocasión únicamente el subconjunto reducido de características seleccionado por el BPSO. Durante esta inferencia final, se extrajeron y evaluaron los indicadores definitivos de eficiencia computacional —específicamente la reducción de latencia temporal y el consumo máximo de memoria RAM—, así como la resiliencia en la detección a través del *F1-Score*. El contraste de estos resultados finales frente al modelo base permitió validar si la arquitectura propuesta logra un equilibrio efectivo entre el despliegue ligero

y la detección precisa exigida en ecosistemas OT (Nasir et al., 2026).

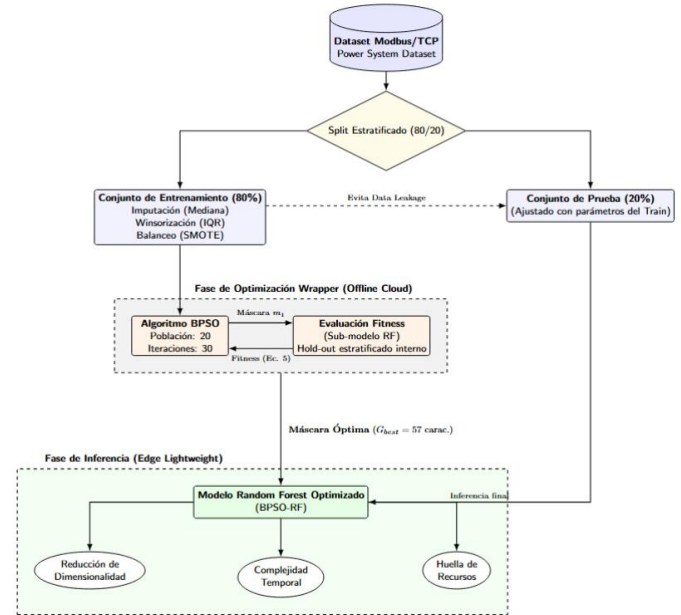


Figura 1

Flujo metodológico para la optimización y despliegue del modelo BPSO-RF

3.4 Comparación Metodológica con Estudios Similares

A diferencia de tendencias recientes en el estado del arte, nuestra investigación eligió un despliegue directo y puro del algoritmo PSO. En contraste, estudios como el de Alsalamah e Ismail (2025) aplican un marco multiobjetivo complejo (MOOIDS-IoT) que delega la selección de características a un Algoritmo Genético (GA) y utiliza el PSO únicamente para el ajuste de hiperparámetros, lo cual incrementa notablemente los tiempos de optimización y entrenamiento. De manera similar, otras propuestas que emplean híbridos profundos impulsados por el Algoritmo de la Luciérnaga (Zayed et al., 2026) o variaciones del Optimizador del Lobo Gris (GWO), exigen sobrecargas computacionales significativas. Se argumenta que nuestro enfoque directo utilizando PSO simplifica la complejidad arquitectónica de forma sustancial; al evitar la hibridación redundante de algoritmos estocásticos, se reduce el costo computacional general del entrenamiento, lo cual favorece activamente su eventual despliegue ágil en nodos de frontera (*Edge*) en la industria.

3.5 Métricas de Evaluación y Operacionalización

Manteniendo la notación matemática y los parámetros del modelo definidos exhaustivamente en la Sección 4, la validación experimental se operalizó a través de dos bloques de evaluación. El primer bloque midió la asertividad predictiva del modelo basándose en los valores extraídos de la Matriz de Confusión: Verdaderos Positivos (*TP*), Verdaderos Negativos (*TN*), Falsos Positivos (*FP*) y Falsos Negativos (*FN*). Las métricas de detección correspondientes se computaron

mediante el siguiente conjunto de ecuaciones estándar en la ciberseguridad industrial:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (2)$$

$$Precision = \frac{TP}{TP + FP} \quad (3)$$

$$Recall = \frac{TP}{TP + FN} \quad (4)$$

$$F1 - Score = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall} \quad (5)$$

Dado el desequilibrio de clases inherente al tráfico en redes OT, si bien se registra la Exactitud (*Accuracy*), el *F1-Score* se establece como la métrica armónica principal para evaluar la asertividad real del modelo frente a amenazas minoritarias.

El segundo bloque, siendo el núcleo crítico para responder a la pregunta de investigación, contempló las Métricas de Eficiencia Computacional. Estos indicadores, que conforman la variable dependiente del estudio, garantizan que el modelo sea viable para su despliegue en la frontera de la red (Edge). De acuerdo con la literatura metodológica contemporánea, su operacionalización se estructuró en tres dimensiones:

1. Reducción de Dimensionalidad

La capacidad del algoritmo BPSO para minimizar el espacio de búsqueda se cuantificó analíticamente mediante la norma L_1 del vector binario de selección, denotada matemáticamente como $\|m\|_1$ (Alqaraghuli & Ibrahim, 2025). En esta expresión, $m \in \{0,1\}^D$ representa la máscara binaria donde 1 indica retención y 0 descarte. Para estandarizar la evaluación, el Ratio de Compresión dimensional $CR(\%)$ se determinó utilizando la formulación adaptada de Gupta et al. (2025), detallada formalmente en la Ecuación 10 de la Sección 4.1.

2. Complejidad Temporal

La eficiencia en los tiempos de respuesta se evaluó mediante un enfoque dual que combina la medición empírica con la reducción de la complejidad algorítmica:

- **Tiempo de entrenamiento (s):** Se cuantificó empíricamente midiendo los segundos exactos de reloj durante la fase de optimización y ajuste del clasificador. Teóricamente, el uso del PSO permite que la carga computacional iterativa dependa asintóticamente del subconjunto reducido $\|m\|_1$ en lugar del espacio original D (Shan, 2025), mitigando la explosión combinatoria.
- **Latencia de inferencia por flujo (ms):** Representada formalmente como t_{lat} (Nasir et al., 2026), indica el tiempo necesario para procesar y clasificar un vector de red individual. Para validar la viabilidad del modelo frente a los requerimientos de tiempo real de la industria, esta latencia se registró en milisegundos netos mediante perfilamiento de hardware (*Hardware*

Profiling).

3. Huella de Recursos

El consumo de hardware subyacente se operacionalizó empíricamente durante las simulaciones de inferencia, monitorizando los límites del entorno operativo mediante rutinas de perfilamiento en tiempo de ejecución:

- **Uso pico de RAM:** Denotado en la literatura como $Mem(\theta)$ (Nasir et al., 2026), se registró el consumo máximo de memoria volátil en megabytes (MB) asignada al proceso. Para asegurar la escalabilidad del modelo en nodos IoT industriales, se evaluó bajo la restricción analítica $\max(0, Mem(\theta) - \theta_{mem})$, donde θ_{mem} define el presupuesto máximo de RAM del dispositivo físico de frontera.

4 Modelos Utilizados

4.1 Algoritmo de Búsqueda Metaheurística: BPSO

El núcleo algorítmico del experimento implementó una variante binaria del PSO (BPSO) como un método de envoltura para la selección automática de características. Cada partícula del enjambre se representó como un vector espacial discreto, donde los valores 1 y 0 determinaron la retención o eliminación de una característica en el subconjunto candidato, logrados típicamente mediante una función de transformación sigmoide (Benmalek & Seddiki, 2025). El desplazamiento estocástico del enjambre en el espacio de búsqueda se rige por la formulación original del PSO Binario introducida por Kennedy y Eberhart (1997); este proceso se define, en primer lugar, por la actualización de la velocidad continua de cada partícula (Ecuación 6). Posteriormente, dicha velocidad se mapea al espacio probabilístico $[0, 1]$ mediante la función sigmoide (Ecuación 7), lo que permite determinar la retención binaria de la característica mediante la regla de actualización de posición (Ecuación 8):

$$v_{ij}(t+1) = w \cdot v_{ij}(t) + c_1 r_{1j}(t)[y_{ij}(t) - x_{ij}(t)] + c_2 r_{2j}(t)[\widehat{y}_{ij}(t) - x_{ij}(t)] \quad (6)$$

Donde $v_{ij}(t)$ y $x_{ij}(t)$ representan la velocidad y la posición actual de la partícula i en la dimensión j durante la iteración t , respectivamente. El parámetro w denota el factor de peso de inercia; c_1 y c_2 son los coeficientes de aceleración cognitiva y social; y $r_{1j}(t)$ junto con $r_{2j}(t)$ son vectores de valores aleatorios extraídos de una distribución uniforme en el intervalo $[0, 1]$. Adicionalmente, $y_{ij}(t)$ constituye la mejor posición personal histórica alcanzada por la partícula (*personal best*), mientras que $\widehat{y}_{ij}(t)$ representa la posición óptima global encontrada por el vecindario o la totalidad del enjambre (*global best*).

Dado que el objetivo es la selección dicotómica de características (retención o eliminación), la velocidad continua resultante no puede aplicarse directamente a la posición espacial. Por lo tanto, se mapea hacia un espacio probabilístico

utilizando una función de transformación sigmoide, expresada en la Ecuación 7:

$$S(v_{ij}(t+1)) = \frac{1}{1 + e^{-v_{ij}(t+1)}} \quad (7)$$

Esta función acota estrictamente la velocidad al rango continuo (0, 1). Finalmente, la actualización de la posición binaria de la partícula —que dictamina empíricamente si la característica j es seleccionada (1) o descartada (0) para el entrenamiento del clasificador— se ejecuta mediante una evaluación estocástica de umbral, definida por la Ecuación 8:

$$X_{ij}(t+1) = \begin{cases} 0, & \text{si } \text{rand}() \geq S(v_{ij}(t+1)) \\ 1, & \text{si } \text{rand}() < S(v_{ij}(t+1)) \end{cases} \quad (8)$$

Donde $\text{rand}()$ es un número pseudoaleatorio muestreado uniformemente en $[0, 1]$. Esta arquitectura matemática garantiza que las características de red que induzcan un mayor rendimiento predictivo atraigan dinámicamente a las partículas hacia estados de activación (1), convergiendo hacia el subconjunto óptimo para el entorno de Tecnologías de Operación.

El motor de evaluación para las partículas fue la Función de Aptitud (*Fitness Function*), definida matemáticamente para lograr un equilibrio estricto entre dos objetivos: maximizar la exactitud predictiva del modelo y minimizar drásticamente el número de características retenidas. En entornos de Tecnologías de Operación es imperativo penalizar la alta dimensionalidad para garantizar un modelo verdaderamente ligero. Basándose en la formulación de Gupta et al. (2025), la idoneidad de cada partícula se evaluó mediante la Ecuación 9, que balancea exactitud y compresión:

$$\text{Fitness} = \alpha \cdot \text{Accuracy} + (1 - \alpha) \cdot \left(\frac{|F| - \|m\|_1}{|F|} \right) \quad (9)$$

Donde *Accuracy* es la exactitud del clasificador Random Forest (RF) evaluando el subconjunto actual. La variable $\|m\|_1$ indica el número de características retenidas y $|F|$ es la dimensionalidad total original. El parámetro α es un peso constante, para este experimento, se fijó empíricamente $\alpha = 0.90$ para priorizar la capacidad de detección. Este último término está vinculado con el Ratio de Compresión (*CR*), calculado mediante la Ecuación 10, el cual cuantifica la eficiencia espacial del modelo:

$$\text{CR}(\%) = \left(1 - \frac{\|m\|_1}{|F|} \right) \cdot 100 \quad (10)$$

Durante el proceso iterativo, las características seleccionadas temporalmente por cada partícula fueron inyectadas en cada ciclo de evaluación en un modelo de Random Forest (RF). Se eligió este clasificador de aprendizaje automático convencional (ML) debido a que ofrece un bajo costo de convergencia durante las iteraciones de la metaheurística, formando una arquitectura altamente eficiente (Gaber et al., 2023).

Para la configuración matemática de la metaheurística descrita, los hiperparámetros de control del BPSO fueron predefinidos de manera heurística, prescindiendo de una fase de optimización empírica exhaustiva (*Offline Tuning*) con el objetivo de minimizar el elevado costo computacional previo al despliegue. En este sentido, se estableció un peso de inercia constante de valor medio ($w = 0.5$). Desde una perspectiva teórica, este ajuste garantiza una transición estable y equilibrada entre la exploración global temprana en el espacio de características y la explotación local intensiva hacia el final de las iteraciones, una dinámica que permite abstraer subconjuntos eficientes en entornos con restricciones operativas (Alqaraghuli & Ibrahim, 2025). Paralelamente, se asignaron valores simétricos e intermedios para el coeficiente de aceleración cognitiva ($c_1 = 1.5$) y el coeficiente de aceleración social ($c_2 = 1.5$). Establecer esta configuración asegura que las partículas confíen equitativamente tanto en su "memoria individual" sobre las mejores posiciones pasadas como en el "conocimiento colectivo del enjambre" al actualizar sus vectores de velocidad (Mohammed et al., 2026). Como argumentan Mohammed et al. (2026), sostener este equilibrio entre los componentes cognitivos y sociales es indispensable para evitar la convergencia prematura en óptimos locales. Si bien se reconoce que la ausencia de un análisis de sensibilidad empírico sobre estos hiperparámetros constituye una limitación del presente diseño experimental, la fijación estática de estos valores estándar —ampliamente validados en la literatura de Inteligencia de Enjambre— garantiza la reproducibilidad exacta del experimento. Este enfoque establece una línea base robusta y transparente, delimitando el alcance del estudio hacia la evaluación de la eficiencia de hardware antes de introducir la complejidad de un autoajuste paramétrico.

4.2 Algoritmo de Clasificación: Random Forest (RF)

Como motor de clasificación y evaluación de la aptitud (*fitness*), se seleccionó el algoritmo *Random Forest* (RF). Este modelo de aprendizaje supervisado se fundamenta en el principio de ensamblaje (*Ensemble Learning*), cuya robustez inherente frente a perturbaciones de datos resulta idónea para entornos industriales basados en el protocolo Modbus/TCP. La elección del RF se justifica empíricamente por su resistencia al sobreajuste (*overfitting*); una propiedad intrínseca demostrada por Breiman (2001), quien establece que a medida que aumenta el número de árboles, el error de generalización de un bosque aleatorio converge hacia un límite probabilístico estricto, impidiendo el sobreajuste. Formalmente, de acuerdo con la definición original del autor, un bosque aleatorio es un clasificador compuesto por una colección estructurada de clasificadores de árboles individuales, denotado como:

$$\{h(x, \theta_k), k = 1, \dots, B\} \quad (11)$$

Donde x representa el vector de entrada (las características de red extraídas por el BPSO), y θ_k conforman una serie de vectores aleatorios independientes e idénticamente distribuidos que dirigen el crecimiento estocástico de cada

árbol k en el bosque B . Tras completarse la fase de entrenamiento, la predicción final del ensamble para dictaminar si un flujo de red corresponde a una operación normal o a un ataque FDIA se determina mediante un sistema donde cada árbol emite un voto unitario (unit vote) por la clase más popular. Este proceso matemático de agregación por voto mayoritario se formaliza como:

$$\hat{y} = \text{moda}\{h_1(x), h_2(x), \dots, h_B(x)\} \quad (12)$$

Para el desarrollo arquitectónico de cada árbol individual $h_k(x)$, el algoritmo emplea la metodología original CART (*Classification and Regression Trees*), permitiendo que los árboles crezcan hasta su tamaño máximo sin poda. En este proceso, el espacio hiperdimensional se particiona iterativamente evaluando subconjuntos de características bajo el criterio de homogeneidad o impureza de Gini. Para un nodo determinado con probabilidad p_i de pertenecer a la clase i dentro de un conjunto de C clases, la impureza se minimiza bajo la siguiente ecuación:

$$I_G(p) = 1 - \sum_{i=1}^C p_i^2 \quad (13)$$

Finalmente, la viabilidad de integrar el RF como estimador interno en el esquema de envoltura (*wrapper*) iterativo del algoritmo BPSO radica en su eficiencia computacional. De acuerdo con el análisis asintótico de Louppe (2015), la complejidad temporal para la construcción del bosque en el caso promedio se encuentra estrictamente acotada por $\mathcal{O}(MK\tilde{N}\log^2\tilde{N})$. En esta formulación, M representa el número total de árboles, K es el subconjunto de características evaluadas aleatoriamente en cada nodo, y $\tilde{N}$ equivale al número de muestras únicas efectivas procesadas tras la aplicación del remuestreo bootstrap. Este comportamiento cuasilineal respecto al volumen de datos asegura una huella de procesamiento ligera, lo que permite que el clasificador sea invocado iterativamente por la metaheurística BPSO para evaluar las funciones de aptitud sin incurrir en cuellos de botella temporales. En consecuencia, se garantiza que el proceso de optimización sea escalable y operacionalmente viable para entornos perimetrales.

4.3 Arquitectura Propuesta: Wrapper BPSO-RF

El modelo propuesto se fundamenta en un enfoque de envoltura (*wrapper*) estructurado en dos capas. En la primera, el algoritmo BPSO actúa como filtro para evaluar y seleccionar iterativamente el subconjunto óptimo de características de red. En la segunda fase, estos datos refinados son procesados por un ensamble Random Forest (RF) para su clasificación. Esta arquitectura aborda la «maldición de la dimensionalidad» inherente al tráfico industrial Modbus/TCP. Al reducir el vector de características mediante BPSO, se disminuye drásticamente la carga computacional, la complejidad espacial y el tiempo de convergencia del RF.

El resultado es un modelo ligero (*lightweight*) que se ajusta a las restricciones computacionales del entorno perimetral (*Edge*) (Gupta et al., 2025; Mohammed et al., 2026), garantizando una latencia de inferencia adecuada para los requerimientos operativos de las redes OT (Nasir et al., 2026). Para consolidar la comprensión de esta integración, el flujo detallado de la arquitectura —desde la inicialización del espacio de búsqueda hasta la obtención del subconjunto de características óptimo— se formaliza en el Algoritmo 1.

Algoritmo 1: Propuesta de Selección de Características BPSO-RF

Input: Conjunto de datos de entrenamiento $\rightarrow D_{train}$, Número de partículas $\rightarrow N$, Máximo de iteraciones $\rightarrow T$, Parámetros del enjambre (w, c_1, c_2, α)

Output: Subconjunto óptimo de características $\rightarrow G_{best}$

1: Fase de Inicialización:

2: **Para** cada partícula i de 1 a N **hacer:**

3: Inicializar posición binaria aleatoria $x_i(0) \in \{0,1\}^d$

4: Inicializar velocidad aleatoria $v_i(0)$

5: Definir mejor posición personal $P_{best} = x_i(0)$

6: **Fin Para**

7: Inicializar mejor posición global G_{best} con el mejor P_{best} de la población

8:

9: Fase de Optimización:

10: **Para** iteración $t = 1$ a T **hacer:**

11: **Para** cada partícula i de 1 a N **hacer:**

12: // 1. Evaluación del Modelo (Clasificación)

13: Decodificar la posición $x_i(t)$ para obtener el subconjunto de características $\|m\|_1$

14: Aplicar validación Hold-out estratificada (80/20) sobre D_{train} filtrado por $\|m\|_1$

15: Entrenar Random Forest (RF) con la partición del 80% y calcular Tasa de Error en el 20% restante

16: Evaluar la penalización por la dimensionalidad retenida en la máscara

17: Calcular *Fitness* actual usando la **Ecuación 5**

18:

19: // 2. Actualización de Memoria

20: **Si** *Fitness actual* > *Fitness*($P_{best, i}$) **entonces:**

21: $P_{best, i} = x_i(t)$

22: **Fin Si**

23: **Fin Para**

24:

25: Actualizar G_{best} identificando el mejor P_{best} de todo el enjambre

26:

27: **Para** cada partícula i de 1 a N **hacer:**

28: // 3. Movimiento del Enjambre

29: Actualizar velocidad continua $v_i(t+1)$ usando

Ecuación 2

30: Calcular probabilidad sigmoide $S(v)$ usando

Ecuación 3

31: Actualizar nueva posición binaria $x_i(t+1)$

usando **Ecuación 4**

32: **Fin Para**

33: **Fin Para**

5 Resultados

El análisis de los resultados experimentales se estructuró evaluando secuencialmente el impacto de la metaheurística de selección (variable independiente) y las métricas resultantes de asertividad predictiva y eficiencia computacional (variable dependiente).

5.1 Impacto en la Reducción de Dimensionalidad

La ejecución del algoritmo metaheurístico BPSO evidenció una notable capacidad para abstraer el espacio de búsqueda. Partiendo de una dimensionalidad original de 128 características extraídas del tráfico Modbus/TCP, el proceso iterativo convergió en un subconjunto óptimo (G_{best}) compuesto por 57 atributos retenidos. Esto se traduce en un Ratio de Compresión (CR) del 55.47%.

Durante la fase de optimización, el algoritmo alcanzó una puntuación de *fitness* final de 0.2012 tras 30 iteraciones.

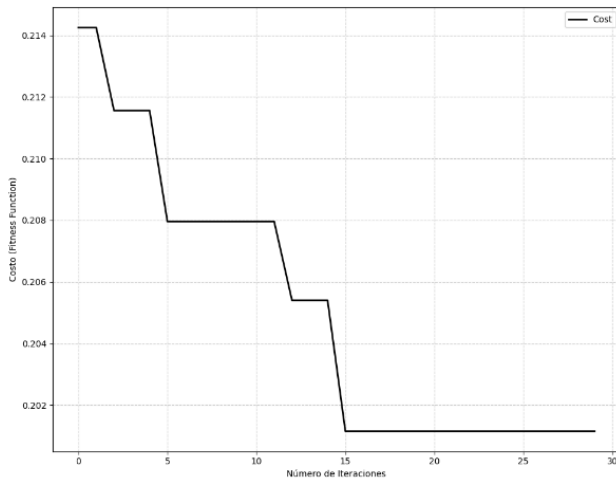


Figura 2
Curva de convergencia del algoritmo BPSO

5.2. Evaluación del Rendimiento Predictivo

Para garantizar que la drástica reducción dimensional no comprometiera la seguridad del entorno OT, se contrastaron las métricas de detección entre ambos modelos. El modelo de control (RF con 128 características) estableció un punto de referencia sólido con un F1-Score de 0.9217 y una exactitud (Accuracy) de 0.9297. Al evaluar la arquitectura propuesta (BPSO-RF) alimentada únicamente con las 57 características seleccionadas, el modelo registró un F1-Score de 0.9166 y una exactitud de 0.9255. Esta variación empírica demuestra una degradación mínima del rendimiento predictivo (una pérdida marginal de aproximadamente 0.51% en el F1-Score). La precisión (Precision) del modelo optimizado se mantuvo sumamente estable (0.9302 frente a 0.9309 del baseline), indicando que la

capacidad para evitar falsas alarmas quedó intacta a pesar de haber eliminado más de la mitad del volumen de datos de entrada.

Tabla 2:
Comparativa de Rendimiento Predictivo

Métrica de Evaluación	Modelo Base	Modelo (BPSO-RF)
Accuracy	0.9296	0.9254
Precision	0.9308	0.9302
Recall	0.9125	0.9032
F1-Score	0.9216	0.9165

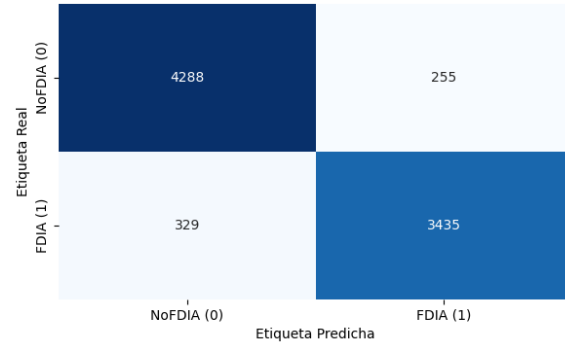


Figura 3
Matriz de Confusión: Grupo de Control

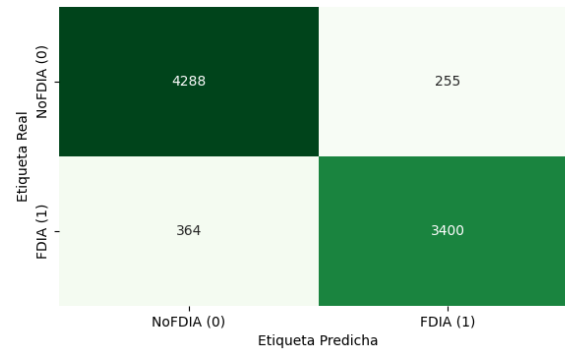


Figura 4
Matriz de Confusión: Grupo Experimental

5.3 Eficiencia Computacional y Viabilidad Operativa

El análisis de las métricas operacionales confirma el impacto directo de la reducción de dimensionalidad en el hardware. Como se detalla en la comparativa de eficiencia, el tiempo de entrenamiento experimentó una contracción significativa del 34.58%, disminuyendo de 25.22 segundos en el modelo de control a 16.50 segundos en el modelo BPSO-RF.

Respecto a la huella de recursos durante la inferencia, el consumo pico de memoria volátil descendió de 4.62 MB a 2.37 MB, logrando una reducción neta del 48.64% en el uso de memoria RAM. Paralelamente, la latencia de inferencia por

flujo se redujo en un 11.01%, pasando de 0.0269 ms a 0.0239 ms.

Tabla 3:
Comparativa de Eficiencia Computacional

Métrica de Evaluación	Modelo Base	Modelo (BPSO-RF)
Dimensionalidad	128 características	57 características
Tiempo de entrenamiento (s)	25.22s	16.50s
Latencia de Inferencia (ms)	0.0268ms	0.0239ms
Uso Pico de RAM (MB)	4.62 MB	2.37 MB

5.4 Aportes y Problemas de Innovación

Aportes Relevantes:

Reducir la huella de RAM a 2.37 MB y mantener una latencia submilisegundo (0.0239 ms) habilita el despliegue directo de estos algoritmos en nodos *Edge* (como PLCs modernos o gateways industriales), validando empíricamente el éxito de la arquitectura propuesta.

Problemas de Innovación y Limitaciones:

A pesar de las marcadas mejoras operativas en la inferencia, los resultados revelan un problema de innovación inherente a la arquitectura *wrapper* propuesta: la alta sobrecarga computacional previa al despliegue. El proceso de selección iterativa del BPSO demandó un tiempo de 2641.76 segundos (aproximadamente 44 minutos) para converger. Esta métrica identifica una limitación crítica; la arquitectura no es apta para reentrenamientos dinámicos o continuos en la frontera de la red, forzando a que la fase de optimización metaheurística deba ejecutarse exclusivamente bajo una modalidad *Offline* en entornos con alta capacidad de procesamiento (Cloud), tal como exige la literatura contemporánea.

6 Discusión

6.1. Análisis de los Hallazgos y Validación de Hipótesis

Los hallazgos empíricos demuestran que la integración del enfoque BPSO-RF logra un balance óptimo entre la compresión dimensional y la asertividad predictiva. Al reducir el vector original de 128 características a un subconjunto de 57 variables, el modelo alcanzó un ratio de compresión del 55.47% frente a la línea base, experimentando una degradación marginal en el F1-Score de apenas 0.51% (descendiendo de 0.9217 a 0.9166). Este comportamiento valida la hipótesis principal del estudio, sugiriendo que la metaheurística descartó eficazmente atributos redundantes del tráfico Modbus/TCP sin comprometer significativamente la capacidad de detección, lo cual se respalda además con métricas estables de exactitud (0.9255) y precisión (0.9302). Un análisis cualitativo del subconjunto óptimo revela que el algoritmo retuvo estratégicamente mediciones críticas de las unidades fasoriales —tales como los ángulos de fase (PA) y magnitudes (PM) de voltaje y corriente en múltiples relés—, variables que constituyen el objetivo directo de alteración para vulnerar los estimadores de estado en los ataques FDIA. Adicionalmente, la preservación conjunta de indicadores lógicos y de red, como

los registros de operación de los relés (*relay_log*) y las alertas del sistema perimetral (*snort_log*), demuestra la capacidad de la metaheurística para correlacionar la desviación anómala de la telemetría puramente eléctrica con los rastros de compromiso a nivel de red.

6.2. Comparación con el estado del arte y trabajos previos

Para validar la solidez de los hallazgos, se procedió a contrastar el desempeño de la arquitectura propuesta BPSO-RF frente a investigaciones contemporáneas de alto impacto. Como paso previo, es imperativo establecer que, debido a la naturaleza heterogénea de los conjuntos de datos, protocolos industriales y metodologías de evaluación en la literatura, una comparación numérica directa presenta limitaciones inherentes. No obstante, para homogeneizar este análisis, se ha seleccionado el F1-Score como métrica de asertividad principal, dada su idoneidad para escenarios de desequilibrio de clases en redes OT, complementada con indicadores de eficiencia de hardware (RAM y Latencia).

En términos de asertividad predictiva, la arquitectura propuesta demuestra una estabilidad competitiva frente a modelos de mayor complejidad. Al compararnos con el modelo de Mohammed et al. (2026), quienes abordaron la detección de ataques FDIA en redes eléctricas inteligentes mediante un híbrido IG-APSO-DNN, se observa que, aunque ellos lograron una reducción dimensional superior (>75%), nuestro enfoque directo con BPSO puro mantuvo una precisión comparable prescindiendo de la alta sobrecarga computacional que su arquitectura de aprendizaje profundo introduce.

El aporte más significativo de esta investigación surge al desplazar el análisis hacia la eficiencia en el hardware perimetral. Un hallazgo crítico reportado por Francis et al. (2026) indica que el 66.7% de los estudios actuales sobre IDS metaheurísticos ignoran por completo la eficiencia computacional real en dispositivos físicos. Nuestra investigación cierra esta brecha al reportar un consumo pico de RAM de tan solo 2.37 MB. Esta cifra representa una ventaja arquitectónica sustancial frente a marcos de trabajo como el de Nasir et al. (2026), cuyo modelo HED-ID reportó consumos de memoria entre 92 y 115 MB, un rango que resulta prohibitivo para nodos IoT industriales ultra-limitados.

Por otro lado, frente al trabajo de Gupta et al. (2025), quienes lograron reducir el uso de RAM en un 57% para detección de malware, nuestro modelo alcanzó una reducción del 48.64%. Sin embargo, se debe destacar que nuestra solución opera sobre tráfico de red nativo OT/SCADA bajo el protocolo Modbus/TCP, a diferencia del enfoque en malware móvil de dicha investigación, lo que valida la versatilidad de la metaheurística BPSO en entornos críticos de manufactura.

6.3. Implicancias en Ciberseguridad OT y Limitaciones

El impacto real de la investigación radica en las implicancias operativas dentro del ecosistema perimetral (*Edge*), un factor crítico en las Tecnologías de Operación (OT). La contracción del espacio de características, además de reducir la huella de memoria, resultó en una latencia de inferencia ultrabaja de

0.0239 ms. Tal como argumentan Nasir et al. (2026) y Gupta et al. (2025), estas métricas empíricas constituyen evidencia significativa que viabiliza el despliegue práctico de esta arquitectura. Bajo estos rangos, el modelo propuesto cumple con los requerimientos técnicos para operar como un Sistema de Detección de Intrusiones (IDS) de naturaleza *Lightweight* en infraestructuras críticas IIoT.

A pesar de los beneficios comprobados en la fase de inferencia, es necesario abordar las limitaciones arquitectónicas asociadas a la carga computacional de la optimización. El tiempo de ejecución requerido de forma aislada por la etapa metaheurística ascendió a 2641.76 segundos, evidenciando la "explosión combinatoria" propia de los métodos de envoltura (*wrapper*). En este contexto, se optó por una validación interna tipo *Hold-out* en lugar de una validación cruzada (*k-fold*) para evaluar la función de aptitud; ejecutar múltiples pliegues por cada evaluación de las partículas habría incrementado exponencialmente este tiempo de optimización, volviéndolo prohibitivo. Este costo se alinea con las limitaciones reportadas por Alsalamah e Ismail (2025), quienes registraron procesos de ajuste de hiperparámetros de hasta 4 horas. Apoyándose en los postulados de Shan (2025), esta masiva demanda de recursos imposibilita la ejecución de un reentrenamiento dinámico directamente en dispositivos perimetrales restringidos. No obstante, al igual que argumentan Tiwari et al. (2024), esta limitación metodológica se compensa con la mejora en la latencia de inferencia, transformando una desventaja de entrenamiento *offline* en una ventaja crítica de respuesta operativa *online*.

6.4. Líneas de Investigación Futura

A partir de la limitación arquitectónica expuesta, se propone formalmente como dirección de investigación futura el diseño de una topología distribuida basada en el paradigma *Cloud-Edge*. En este esquema, el ajuste de hiperparámetros (*Offline Tuning*) y la ejecución pesada del algoritmo BPSO deberán delegarse en su totalidad a una capa *Cloud* de alta capacidad. Una vez completada la optimización, se proyecta transferir únicamente la máscara óptima de características (las 57 variables seleccionadas) y los pesos estáticos del *Random Forest* hacia el nodo *Edge* para su operacionalización. Esta delegación garantizará una detección de amenazas perimetral en tiempo real, manteniendo intacta la ligereza del sistema en el entorno industrial de despliegue final. Complementariamente, para superar la falta de un análisis de sensibilidad empírico abordada en este estudio, se proyecta la implementación de variantes de Enjambre Auto-Adaptativo (Self-Adaptive PSO). Esto permitirá el ajuste dinámico de la inercia (w) y los coeficientes de aceleración (c_1, c_2) en tiempo de ejecución, asegurando que el modelo alcance el óptimo global absoluto del espacio dimensional sin comprometer la reproducibilidad del algoritmo.

7 Conclusiones

La presente investigación permitió validar de manera concluyente la hipótesis principal: la integración de la Optimización por Enjambre de Partículas Binario (BPSO) como motor de selección de características mejora

sustancialmente la eficiencia computacional en sistemas de detección de intrusiones para entornos de Tecnologías de Operación (OT), sin comprometer la asertividad predictiva necesaria para la protección de infraestructuras críticas.

En términos de optimización dimensional, la arquitectura propuesta logró una reducción del 55.47% en el espacio de características, abstrayendo un subconjunto de 57 variables críticas a partir del vector original de 128 atributos. Este avance técnico demuestra la eficacia del enfoque metaheurístico de envoltura para procesar tráfico Modbus/TCP, identificando y reteniendo las características con mayor ganancia de información para la detección de ataques de inyección de datos falsos (FDIA).

La relevancia científica del estudio radica en la validación empírica del impacto en el hardware. Los resultados demuestran que la reducción de la carga computacional se traduce en una mejora del 48.64% en el consumo pico de memoria RAM (alcanzando 2.37 MB) y una contracción del 11.01% en la latencia de inferencia (0.0239 ms). Estas métricas confirman su aplicabilidad práctica inmediata para el despliegue en dispositivos de borde (*Edge*) y nodos industriales.

A pesar de la degradación marginal en el desempeño predictivo (una pérdida de solo 0.51% en el F1-Score respecto al modelo base), la robustez del clasificador Random Forest se mantuvo estable, garantizando una detección asertiva. No obstante, se concluye que el elevado costo temporal de la fase de optimización (2641.76 segundos) actúa como un factor limitante para actualizaciones dinámicas en el perímetro. Por tanto, la investigación establece que la viabilidad operativa de estos sistemas depende de una división metodológica estricta entre una fase de entrenamiento y optimización *Offline* en entornos de alta capacidad, y una fase de ejecución ligera en el entorno industrial de despliegue final.

Finalmente, este estudio cierra la brecha identificada en la literatura respecto a la falta de mediciones de hardware en sistemas IDS bioinspirados. Los hallazgos proporcionan un marco de referencia sólido para el diseño de arquitecturas de ciberseguridad sostenibles y escalables, orientando el desarrollo futuro hacia topologías distribuidas que maximicen la protección de los Sistemas de Control Industrial frente a amenazas contemporáneas.

Referencias

- Alsalamah, H. A., & Ismail, W. N. (2025). A swarm-based multi-objective framework for lightweight and real-time IoT intrusion detection. *Mathematics*, 13(15), Artículo 2522. <https://doi.org/10.3390/math13152522>
- Alqaraghuli, A. M., & Ibrahim, A. (2025). Vampire squid optimization algorithm for energy-efficient intrusion detection in cyber-physical networks. *Discover Computing*, 28(1), Artículo 9824. <https://doi.org/10.1007/s10791-025-09824-7>
- Benmalek, M., & Seddiki, A. (2025). Particle swarm optimization-enhanced machine learning and deep learning techniques for Internet of Things intrusion detection. *Data*

Science and Management, 8(4), 423–435. <https://doi.org/10.1016/j.dsm.2025.02.005>

Borges Hink, R. C., Beaver, J. M., Buckner, M. A., Morris, T., Adhikari, U., & Pan, S. (2014). Machine learning for power system disturbance and cyber-attack discrimination. En *2014 7th International Symposium on Resilient Control Systems (ISRCS)* (pp. 1–8). IEEE. <https://doi.org/10.1109/ISRCS.2014.6900095>

Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>

Francis, G. T., Souri, A., & Inanç, N. (2026). A systematic review of metaheuristic based feature selection strategies for cyber-attack detection in the IIoT. *Artificial Intelligence Review*, 59, Artículo 73. <https://doi.org/10.1007/s10462-025-11473-7>

Gaber, T., Awotunde, J. B., Folorunso, S. O., Ajagbe, S. A., & Eldesouky, E. (2023). Industrial Internet of Things intrusion detection method using machine learning and optimization techniques. *Wireless Communications and Mobile Computing*, 2023, Artículo 3939895. <https://doi.org/10.1155/2023/3939895>

Gupta, G. P., Kumar, P., & Aljuhani, A. (2025). An efficient malware detection framework for enhancing software security in resource-constrained systems. *IEEE Internet of Things Journal*, 12(21), 44897–44908. <https://doi.org/10.1109/JIOT.2025.3596313>

Kennedy, J., & Eberhart, R. C. (1997). A discrete binary version of the particle swarm algorithm. En *1997 IEEE International Conference on Systems, Man, and Cybernetics. Computational Cybernetics and Simulation* (Vol. 5, pp. 4104–4108). IEEE. <https://doi.org/10.1109/ICSMC.1997.637339>

Louppe, G. (2015). *Understanding random forests: From theory to practice*. arXiv. <https://arxiv.org/abs/1407.7502>

Mohammed, S. H., Jit Singh, M. S., Al-Jumaily, A., Tariqul Islam, M., Islam, M. S., Alenezi, A. M., Alawad, M. A., & Alsoufi, M. A. (2026). IG-APSO-DNN: Deep learning intrusion detection model to detect false data injection attacks in smart grids. *Ad Hoc Networks*, 180, Artículo 104053. <https://doi.org/10.1016/j.adhoc.2025.104053>

Nasir, K., Badri, S. K., Alghazzawi, D. M., Alghamdi, M. Y., Alkhozae, M., Almakky, A., Alhazmi, R. M., & Asghar, M. Z. (2026). HED-ID: An edge-deployable and explainable intrusion detection system optimized via metaheuristic learning. *Scientific Reports*, 16(1), Artículo 32183. <https://doi.org/10.1038/s41598-025-32183-8>

Shan, L. (2025). (IoT) Network intrusion detection system

using optimization algorithms. *Scientific Reports*, 15(1), Artículo 4638. <https://doi.org/10.1038/s41598-025-04638-5>

Shees, A., Tariq, M., & Sarwat, A. I. (2024). Cybersecurity in Smart Grids: Detecting False Data Injection Attacks Utilizing Supervised Machine Learning Techniques. *Energies*, 17(23), 5870. <https://doi.org/10.3390/en17235870>

Tiwari, R. S., Lakshmi, D., Das, T. K., Tripathy, A. K., & Li, K.-C. (2024). A lightweight optimized intrusion detection system using machine learning for edge-based IIoT security. *Telecommunication Systems*, 87(3), 605–624. <https://doi.org/10.1007/s11235-024-01200-y>

Zayed, S. M., Alshathri, S., & El-Shafai, W. (2026). Firefly algorithm optimized hybrid deep learning framework for intrusion detection in IoT environments. *International Journal of Computational Intelligence Systems*, 19(1), Artículo 1178. <https://doi.org/10.1007/s44196-026-01178-2>

Disponibilidad de los Datos

El conjunto de datos generado y analizado durante el presente estudio está disponible en el repositorio Figshare y se puede acceder en <https://doi.org/10.6084/m9.figshare.27798429>.